

SISTEMA DE CONTROL DE INVENTARIO

Pinturas y Ferretería Taximaroa

Reporte del Proyecto Integrador – Fase Final

Empresa	Pinturas y Ferretería Taximaroa
Giro	Venta y distribución de productos de ferretería y pintura
Área	Control de inventario y registro de ventas
Lenguaje	Python
Versión	Actualizada con avances de Fase Final

1. Problemática de la empresa

La empresa lleva el control de sus productos de manera manual mediante hojas de papel o anotaciones. Esto puede provocar errores en las cantidades disponibles, dificultad para detectar productos con pocas unidades y pérdida de tiempo al revisar o actualizar existencias. El reporte original identifica precisamente estos problemas como la situación que el sistema busca resolver. ■filecite■turn0file0■L10-L27■

2. Propuesta de solución

Se propone un sistema de control de inventario desarrollado en Python. La versión original permitía registrar productos, mostrar inventario, realizar ventas y salir del sistema. ■filecite■turn0file0■L28-L40■ La versión actual conserva esas funciones y las amplía con archivos de texto, lectura y escritura de información, captura de fecha, funciones, manejo de excepciones y una estructura de menú mediante matriz.

3. Objetivo general

Desarrollar un sistema sencillo para administrar las existencias de Pinturas y Ferretería Taximaroa, permitiendo registrar productos, modificar cantidades, realizar ventas y conservar registros de las operaciones en archivos de texto.

4. Objetivos específicos

- Registrar tres productos y sus cantidades iniciales.
- Agregar existencias al inventario.
- Realizar ventas verificando las existencias disponibles.
- Mostrar el inventario actual y el contador de ventas.
- Crear y consultar archivos de texto.
- Registrar la fecha de las operaciones.
- Controlar errores de entrada y de archivos.
- Organizar el programa mediante funciones.

5. Estructura actual del programa

El programa está dividido en funciones: crear_archivos(), cargar(), capturar_fecha(), guardar_archivo(), leer_archivo(), agregar_producto(), realizar_venta(), revisar_inventario() y mostrar_menu(). Después se encuentra el programa principal, que llama a estas funciones y controla el menú mediante un ciclo while.

6. Funciones principales

Función	Qué realiza
crear_archivos()	Crea inventario.txt, ventas.txt, productos.txt y usuarios.txt.
cargar()	Muestra una barra de carga utilizando for y time.sleep().
capturar_fecha()	Solicita día, mes y año y los guarda en la tupla Fecha.
guardar_archivo()	Agrega fecha y datos a un archivo usando el modo a.
leer_archivo()	Permite seleccionar uno de los cuatro archivos y leerlo.
agregar_producto()	Aumenta la cantidad de un producto y registra el movimiento.
realizar_venta()	Resta unidades, aumenta ventas y registra la venta.
revisar_inventario()	Muestra productos, cantidades y contadores.
mostrar_menu()	Presenta las opciones mediante una matriz.

7. Variables principales

producto1, producto2 y producto3 son strings que almacenan nombres. cantidad1, cantidad2 y cantidad3 son enteros que representan existencias. ventas cuenta las ventas realizadas y contador registra la cantidad de productos. opcion guarda la selección del menú; producto y cantidad se utilizan en las operaciones. La versión inicial del proyecto ya definía estas variables para el inventario y las ventas. ■filecite■turn0file0■L41-L57■

8. Persistencia mediante archivos

El sistema crea cuatro archivos externos: inventario.txt, ventas.txt, productos.txt y usuarios.txt. En crear_archivos() se usa el modo w para crear el contenido inicial. En guardar_archivo() se usa el modo a para agregar información al final sin borrar los registros anteriores. En leer_archivo() se utiliza el modo r para consultar la información. De esta manera, los datos de las operaciones pueden permanecer almacenados fuera de las variables del programa.

9. Captura de fecha

El usuario escribe la fecha con formato día/mes/año. El método split('/') separa las tres partes, int() las convierte a números y después se construye exactamente la estructura Fecha = día, mes, año. La fecha se pasa como argumento a las funciones que guardan información y se escribe junto con cada operación.

10. Manejo de excepciones

El programa utiliza try-except para evitar cierres inesperados. ValueError controla entradas que deberían ser números. FileNotFoundError controla archivos inexistentes. PermissionError controla problemas de permisos. OSError controla otros errores relacionados con el sistema de archivos. Esto hace que el programa informe el problema al usuario en lugar de terminar abruptamente.

11. Menú como matriz

El menú se construye como una lista de listas: cada fila contiene el número de la opción y su descripción. El for recorre cada fila y muestra sus dos elementos. Actualmente las opciones son: 1) Agregar producto, 2) Venta, 3) Revisar inventario, 4) Leer archivos, 5) Guardar reporte y 6) Salir.

12. Registro de productos

El ciclo `for range(1, 4)` permite realizar tres registros. Dependiendo del valor de `i`, se guarda el nombre y cantidad en `producto1/cantidad1`, `producto2/cantidad2` o `producto3/cantidad3`. La cantidad se convierte con `int()` porque `input()` devuelve texto.

13. Agregar producto

El usuario selecciona un producto y una cantidad. Se valida que la cantidad sea mayor que cero. Después, mediante `if/elif/else`, se aumenta la variable de cantidad correspondiente. La operación se transforma en un texto y se guarda en `inventario.txt` junto con Fecha.

14. Realizar venta

El usuario selecciona el producto y la cantidad. Primero se verifica que la cantidad sea positiva. Después se comprueba que no sea mayor que el inventario disponible. Si la venta es válida, se resta la cantidad y se incrementa ventas en uno. Finalmente se registra la operación en `ventas.txt`.

15. Lectura de archivos

`leer_archivo()` muestra una lista de cuatro archivos. El usuario selecciona un número. El programa utiliza `opcion - 1` porque las listas empiezan a contar desde índice 0. Después abre el archivo con modo `r` y utiliza `read()` para obtener todo su contenido.

16. Flujo general del sistema

1. Se crean los archivos iniciales.
2. Se solicita el usuario o nickname.
3. Se muestra una bienvenida dinámica.
4. Se ejecuta la barra de carga.
5. Se captura la fecha.
6. Se registran tres productos.
7. Se guardan los productos en productos.txt.
8. Se muestra el menú principal.
9. Se ejecuta la operación seleccionada.
10. Las operaciones se guardan en archivos cuando corresponde.
11. El menú continúa hasta seleccionar Salir.

17. Cambios respecto al prototipo de la Fase I

El prototipo original manejaba directamente las variables de productos, cantidades, ventas y el menú. El pseudocódigo inicial también contemplaba las operaciones básicas de agregar, vender y revisar inventario.

La versión actual reorganiza esas operaciones en funciones y agrega persistencia mediante archivos, fecha estructurada, lectura de archivos y excepciones.

18. Tabla de avances

Elemento	Fase I	Versión actual
Registro de productos	Sí	Sí
Inventario	Sí	Sí
Ventas	Sí	Sí
Menú	Sí	Sí, ahora como matriz
Funciones	No como estructura principal	Sí
Archivos externos	No	Sí
Lectura de archivos	No	Sí
Escritura/anexado	No	Sí
Fecha en registros	No	Sí
Excepciones	Básicas/no integradas	Sí
PDB	No	Importado y listo para debugging

19. Depuración técnica

El código importa el módulo pdb para realizar debugging. Para documentar una sesión de depuración se puede colocar temporalmente `pdb.set_trace()` dentro de una operación, por ejemplo en `realizar_venta()`. Desde el prompt (Pdb) se pueden consultar variables con `p nombre_variable`, avanzar con `n` y continuar con `c`. La finalidad es comprobar los valores de cantidades, producto y ventas y localizar errores de lógica.

20. Resultado esperado

El sistema permite que el negocio tenga una forma sencilla de registrar productos y controlar sus existencias. Las entradas aumentan el inventario, las ventas lo reducen y las ventas válidas incrementan el contador. Además, las

operaciones guardadas en archivos pueden consultarse posteriormente.

21. Conclusión

El proyecto evolucionó de un prototipo de inventario a una versión más completa y organizada. Se conserva el problema original de la empresa y la solución propuesta, pero ahora el sistema incorpora funciones, persistencia en archivos, lectura y escritura, fecha estructurada y manejo de excepciones. Estos cambios permiten que el programa sea más organizado, reutilizable y capaz de conservar información de las operaciones.

22. Nota sobre requisitos pendientes

El código actual ya contiene la base para el requisito de debugging mediante PDB. Sin embargo, el control específico de inactividad de 10 minutos solicitado en la consigna todavía no está integrado en el código mostrado. También conviene verificar que el programa final utilice `pdb.set_trace()` durante la evidencia de depuración. Estas partes deben agregarse antes de considerar que los diez requisitos están completamente implementados.

23. Guion breve para explicar el proyecto

Puedes explicar el proyecto así: 'Mi proyecto busca solucionar el problema del control manual de inventario de una ferretería. Primero el programa solicita un usuario, muestra una bienvenida y carga el sistema. Después registra la fecha y tres productos con sus cantidades. El menú está construido como una matriz y funciona dentro de un `while`. Desde ahí puedo agregar inventario, realizar ventas, revisar existencias, leer archivos y guardar reportes. Las operaciones de inventario y ventas se almacenan en archivos de texto y llevan la fecha de operación. También utilizo `try-except` para controlar errores y el módulo PDB para depurar el programa.'